

JOB PORTAL MANAGEMENT SYSTEM

School of Computing

**Bachelor of Technology
in
Computer Science & Engineering**

By

BOGGULA YUVARAJ REDDY (VTU25882)

10211CS224

Full Stack Application Development

Slot : E3-B19



**DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
SCHOOL OF COMPUTING**

**VEL TECH RANGARAJAN Dr.SAGUNTHALA R&D
INSTITUTE OF SCIENCE AND TECHNOLOGY
(Deemed to be University Estd u/s 3 of UGC Act, 1956)
CHENNAI 600 062, TAMILNADU, INDIA**

May, 2026

BONAFIDE CERTIFICATE

It is certified that the work contained in the Full Stack Application Development report titled "Job Portal Management System" by BOGGULA YUVARAJ REDDY (VTU25882) has been carried out in Winter semester of Academic year 2025-2026(WS2526).

Signature of Faculty

Dr. X.Arogya Presskila

Assistant Professor (Senior Grade)

Computer Science & Engineering

School of Computing

Vel Tech Rangarajan Dr.Sagunthala R&D

Institute of Science and Technology

May, 2026

Signature of Course Coordinator

Dr. M.Sumithra

Assistant Professor (Senior Grade)

Computer Science & Engineering

School of Computing

Vel Tech Rangarajan Dr.Sagunthala R&D

Institute of Science and Technology

May, 2026

DECLARATION

I declare that this written submission represents my ideas in my own words and where others' ideas or words have been included, we have adequately cited and referenced the original sources. I also declare that i have adhered to all principles of academic honesty and integrity and have not misrepresented or fabricated or falsified any idea/data/fact/source in our submission. I understand that any violation of the above will be cause for disciplinary action by the Institute and can also evoke penal action from the sources which have thus not been properly cited or from whom proper permission has not been taken when needed.

(Boggula Yuvaraj Reddy)

Date:06-05-2026

ABSTRACT

The project presents the design and development of a modern, Job Portal Management System aimed at bridging the gap between job seekers and employers. Built with a robust Spring Boot RESTful backend, a MySQL database, and a dynamic React frontend featuring a bold Neo-Brutalist design, the platform offers a highly responsive and engaging user experience. The system incorporates role-based access control, catering to distinct personas: Students, Employers, and Administrators. Key features include an intuitive job search and application tracking mechanism for students, alongside an advanced Employer Dashboard equipped with applicant management and an AI-driven interview question generator. To ensure security and system integrity, the application implements real-time SMTP-based OTP email verification and JWT authentication. By streamlining the recruitment workflow and providing advanced filtering and communication tools, this portal serves as a comprehensive, secure, and user-centric solution for modern employment networking.

Keywords: Job Portal, Full-Stack Development, React, Spring Boot, Role-Based Access Control, Recruitment Automation, Neo-Brutalist UI, Secure Authentication

LIST OF FIGURES

1.1	Framework and Architecture	3
3.1	Backend Execution	11
4.1	System Architecture Diagram	14
4.2	DataFlow Diagram	15
5.1	Login page for Employer and Student Authentication	17
5.2	Student Dashboard	18
5.3	Admin Dashboard	19

LIST OF TABLES

2.1	Feature Comparison with Market Leaders	6
7.1	Mapping of Project to Complex Engineering Problem (CEP)	21
7.2	Mapping of Project to Complex Engineering Activities (CEA) . . .	22
7.3	SDG Mapping for the Project	22

LIST OF ACRONYMS AND ABBREVIATIONS

API	Application Programming Interface
ATS	Applicant Tracking System
CI	Continuous Integration
CSS	Cascading Style Sheets
HTML	HyperText Markup Language
HTTP	HyperText Transfer Protocol
JWT	JSON Web Token
ORM	Object Relational Mapping
OTP	One-Time Password
RBAC	Role-Based Access Control
REST	Representational State Transfer
SMTP	Simple Mail Transfer Protocol
SQL	Structured Query Language
UI	User Interface
UX	User Experience

TABLE OF CONTENTS

	Page.No
ABSTRACT	iii
LIST OF FIGURES	iv
LIST OF TABLES	v
LIST OF ACRONYMS AND ABBREVIATIONS	vi
1 INTRODUCTION	1
1.1 Introduction	1
1.2 Aim of the Project	2
1.3 Scope of the Project	2
1.4 Framework	3
2 SURVEY OF SIMILAR APPLICATION IN MARKET	4
2.1 Global Job Portals	4
2.2 Company Insight Platforms	4
2.3 Indian Job Portals	4
2.4 AI-Driven Recruitment Platforms	5
2.5 Identified Research Gap	5
2.6 Comparison of Existing Systems	5

2.7	Need for Proposed System	6
3	FRAMEWORK DESCRIPTION	8
3.1	Frontend Implementation	8
3.1.1	Environment Setup	8
3.1.2	Structure of the Project	8
3.1.3	Execution Procedure	9
3.2	Backend Implementation	10
3.2.1	Environment Setup	10
3.2.2	Structure of the Project	10
3.2.3	Execution Procedure	11
3.3	Database Implementation	11
3.3.1	Database Configuration	11
3.3.2	Schema Structure	12
3.3.3	Tables Created	12
4	METHODOLOGIES	14
4.1	Project Architecture	14
4.2	DataFlow Diagram	15
4.3	Module 1:User Management Module	16
4.4	Module 2:Job Management Module	16
4.5	Module 3:Status Alert Module	16
5	RESULTS AND DISCUSSIONS	17
6	CONCLUSION AND FUTURE ENHANCEMENTS	20
6.1	Conclusion	20
6.2	Future Enhancements	20

7 ADDRESSING COMPLEX ENGINEERING PROBLEM AND SDG 21

7.1 Mapping of the Project to Complex Engineering Problem (CEP) . . 21

7.2 Mapping of the Project to Complex Engineering Activities (CEA) . 22

7.3 SDG Mapping (CEP Section) 22

References 23

Chapter 1

INTRODUCTION

1.1 Introduction

The rapid evolution of the digital landscape has fundamentally revolutionized the global recruitment process, transitioning it from static, fragmented methods to dynamic, data-driven ecosystems. Traditional approaches to job hunting and talent acquisition are often hindered by time-consuming manual screenings, lack of real-time communication, and administrative bottlenecks that frustrate both recruiters and job seekers. To address these challenges, the Job Portal Management System is designed as a centralized, high-performance platform that seamlessly aligns the needs of candidates with modern employers. By integrating a robust Spring Boot backend and a high-performance React frontend with a Neo-Brutalist design, the system ensures a secure, scalable, and frictionless experience. It also incorporates advanced features such as automated application tracking, AI-driven question generation, role-based access control, and real-time notifications to optimize the hiring lifecycle and enhance efficiency.

1.2 Aim of the Project

The primary aim of this project is to design and develop a secure, full-stack Job Portal Management System that facilitates seamless interaction between job seekers and recruiters. It focuses on providing a high-performance environment with role-based dashboards, real-time security verification (SMTP OTP), and AI-driven productivity tools to simplify the modern hiring process.

1.3 Scope of the Project

The scope of this project encompasses several critical functional areas that collectively enhance the recruitment process. It includes secure onboarding through a robust authentication system that leverages JWT and real-time SMTP-based email OTP verification to ensure user identity and data protection. The system provides centralized job management, enabling employers to efficiently create, update, and oversee job listings while also accessing applicant statistics through a dedicated dashboard. For job seekers, it facilitates seamless job discovery and tracking, allowing students to search, filter, and apply for jobs while monitoring their application status in real time. Additionally, the platform incorporates automated recruitment assistance by integrating an AI-powered tool that generates role-specific interview questions for employers. Furthermore, it features an Applicant Tracking System (ATS), offering a specialized interface for employers to manage candidate profiles, review resumes, and update application statuses such as shortlisting or rejection, thereby streamlining the entire hiring workflow.

1.4 Framework

The project is developed using a three-tier architecture to ensure scalability, security, and clear separation of concerns. The presentation layer frontend is built using React.js with Vite, styled using custom Vanilla CSS following a Neo-Brutalist design approach, and utilizes React Router DOM for seamless navigation. The application or logic layer (backend) is implemented using Java Spring Boot, incorporating Spring Security and JWT for secure authentication and authorization, and exposes functionality through RESTful web services. The data layer is managed using a MySQL database, with Spring Data JPA (Hibernate) serving as the ORM to efficiently handle database interactions and relationships.

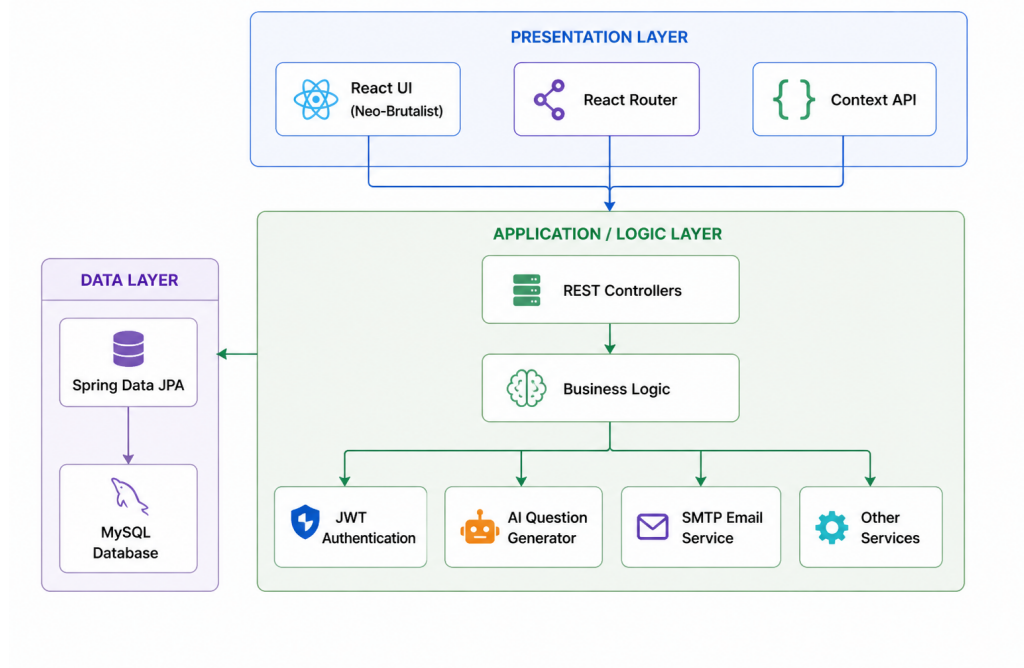


Figure 1.1: Framework and Architecture

Figure 1.1 illustrates the framework architecture of the Job Portal Management System. It shows how different layers of the system interact. The architecture ensures modularity, scalability, and smooth system communication.

Chapter 2

SURVEY OF SIMILAR APPLICATION IN MARKET

2.1 Global Job Portals

Leading platforms like LinkedIn and Indeed dominate the global recruitment space by offering extensive job listings, professional networking, and simplified application processes. These platforms focus on connecting recruiters and candidates at scale with user-friendly interfaces and wide reach.

2.2 Company Insight Platforms

Glassdoor stands out by providing detailed insights into company culture, salary structures, and interview experiences. This helps candidates make informed career decisions beyond just applying for jobs.

2.3 Indian Job Portals

In India, platforms such as Naukri.com and Internshala are widely used. They cater to both experienced professionals and fresh graduates, offering targeted job

listings, internships, and skill-based opportunities.

2.4 AI-Driven Recruitment Platforms

Modern platforms like Monster and ZipRecruiter integrate advanced features such as resume parsing and AI-driven job recommendations to improve hiring efficiency and candidate matching.

2.5 Identified Research Gap

Despite these advancements, most existing systems focus on specific aspects such as job listings, networking, or analytics individually. There remains a gap in delivering a unified platform that integrates secure authentication, AI-powered recruitment assistance, modern UI design, and a comprehensive applicant tracking system—an area this project aims to address.

2.6 Comparison of Existing Systems

To better understand the competitive landscape, the following table compares established market leaders with our proposed Job Portal Management System based on key technical and functional parameters.

Feature	LinkedIn	Naukri.com	Internshala	Proposed System
Job Application	High	High	High	High
AI Interview/Question Bot	No	No	No	Yes
Neo-Brutalist UI Design	No	No	No	Yes
Integrated Support Chatbot	Limited	No	No	Yes
SMTP Email Verification	Yes	Yes	Yes	Yes
Real-time Analytics	Paid	Basic	Basic	Built-in

Table 2.1: Feature Comparison with Market Leaders

Table 2.1 presents a comparison of existing job portal platforms with the proposed system based on key features. It highlights the additional functionalities such as AI-driven interview support, improved user interface design, and integrated real-time communication.

2.7 Need for Proposed System

The necessity for the proposed Job Portal Management System arises from several critical limitations in current market offerings:

- **Cognitive UX Improvement:** Most current portals utilize cluttered, advertisement-heavy interfaces that increase user fatigue. The proposed system employs a bold Neo-Brutalist design that prioritizes readability and essential navigation.
- **AI-Assisted Screening:** Traditional portals act as passive bulletin boards. The

proposed system introduces an AI Question Generator and Interview Bot, enabling employers to screen candidates effectively and helping students prepare for job roles.

- **Streamlined Communication:** There is a significant need for real-time engagement. By integrating a chatbot widget and automated SMTP-based notifications, the system ensures timely updates and continuous communication.
- **High-Security Standards:** By leveraging JWT-based stateless authentication and secure file handling for resumes, the system provides a safer environment for managing sensitive professional data.
- **Unified Role Management:** The system provides specialized dashboards for both Employers and Students within a unified architecture, eliminating the need for external tracking tools.

Chapter 3

FRAMEWORK DESCRIPTION

3.1 Frontend Implementation

3.1.1 Environment Setup

The frontend of the Job Portal Management System is developed using a modern JavaScript-based technology stack to ensure high performance and responsiveness. It is built using React.js along with Vite as the development and build tool. React is used to create dynamic and reusable user interface components, while Vite enables fast development through efficient Hot Module Replacement. Node.js and npm are used as the runtime environment and package manager to manage dependencies. Additional libraries such as Axios are used for handling API requests, Framer Motion is used to implement smooth animations, and Lucide-React provides clean and scalable icons. The development is carried out using Visual Studio Code, which ensures code quality with tools like ESLint and Prettier. The frontend is executed in a web browser and communicates with the backend through REST APIs.

3.1.2 Structure of the Project

The Job Portal Management System follows a well-organized frontend folder structure to maintain the application efficiently. The React JS application is divided

into multiple folders and files based on functionality. This structure enables easy navigation, better code management, and improved maintainability.

The main project structure includes folders for components, pages, assets, and services. Each feature of the application such as student dashboard, employer dashboard, authentication, and job listings is developed as a separate React component, ensuring modularity and reusability.

- **components/** – Contains reusable UI elements such as navbar, chatbot widget, forms, and buttons.
- **pages/** – Includes major application views such as landing page, login page, student dashboard, and employer dashboard.
- **assets/** – Stores static resources like images, icons, and fonts.
- **services/** – Handles API communication with the backend using Axios.
- **App.jsx** – Main component responsible for routing using React Router.
- **main.jsx** – Entry point of the React application that initializes the app.
- **styles.css** – Contains global styling following the Neo-Brutalist design approach.

3.1.3 Execution Procedure

To execute the frontend application, open the terminal and navigate to the frontend project directory. Install all required dependencies using the command `npm install`. After successful installation, run the application using `npm run dev`. This starts the development server provided by Vite. The application can then be accessed through a web browser at `http://localhost:8080` or the specified port. The frontend will interact with the backend server to fetch and display dynamic data.

3.2 Backend Implementation

3.2.1 Environment Setup

The backend of the Job Portal Management System is developed using Java and the Spring Boot framework to ensure scalability and security. Java Development Kit (JDK) 17 is used as the programming environment. Spring Boot simplifies the development of RESTful web services and provides built-in features for dependency injection and configuration. Maven is used for project build and dependency management. MySQL is used as the relational database to store user data, job postings, and application details. Postman is used for testing API endpoints during development. The backend is designed to handle authentication, authorization, and data processing efficiently.

3.2.2 Structure of the Project

The backend project follows an organized folder structure to manage server-side functionality efficiently. The structure helps separate different functionalities such as routing, data handling, and server configuration. This improves readability, maintainability, and scalability of the project.

- **routes/** – Contains API routes for handling client requests.
- **models/** – Defines the data structure and handles database-related logic.
- **controllers/** – Implements backend business logic and processes requests.
- **config/** – Stores configuration settings such as database connections and environment variables.
- **server.js** – Main server file used to initialize and run the backend application.

- **package.json** – Contains project dependencies and scripts required for running the application.

placeins

3.2.3 Execution Procedure

To run the backend application, ensure that the MySQL server is running and the required database is created. Configure the application.properties file with the correct database URL, username, and password. Build the project using the command mvnw clean install. After building, run the application using mvn spring-boot:run or by executing the main class. The backend server will start on port 8081 and will be ready to handle requests from the frontend.

```
(base) yuva@yuvarajreddys-MacBook-Air job portal project % ./mvnw spring-boot:run*
[INFO] -----< com.example:demo >-----
[INFO] Building 0.0.1-SNAPSHOT
[INFO] from pom.xml
[INFO] -----[ jar ]-----
[INFO] >>> spring-boot:4.0.6:run (default-cli) > test-compile @ demo >>>
[INFO] --- resources:3.3.1:resources (default-resources) @ demo ---
[INFO] Copying 1 resource from src/main/resources to target/classes
[INFO] Copying 10 resources from src/main/resources to target/classes
[INFO] --- compiler:3.14.1:compile (default-compile) @ demo ---
[INFO] Nothing to compile - all classes are up to date.
[INFO] --- resources:3.3.1:testResources (default-testResources) @ demo ---
[INFO] skip non existing resourceDirectory /Users/yuva/Documents/job portal proje
ct/src/test/resources
```

Figure 3.1: Backend Execution

Figure 3.1 represents the frontend and backend architecture of the proposed system, highlighting the tools and technologies used in both layers.

3.3 Database Implementation

3.3.1 Database Configuration

The Job Portal Management System utilizes MySQL as its primary relational database management system (RDBMS). Connectivity between the Java backend

and the database is established using Spring Data JPA and Hibernate ORM, with configuration parameters defined in the `application.properties` file. This setup ensures robust data persistence, secure connection pooling, and efficient transaction management. The database is responsible for storing all critical application data, including user credentials, job post details, application statuses, and resume metadata.

3.3.2 Schema Structure

The database schema is designed using an Entity-Relationship (ER) model that ensures data integrity and minimizes redundancy. The core entities User, Job, Application, and Resume are linked through strategic relationships. Primary keys are used for unique identification, while foreign keys are implemented to maintain referential integrity across the platform. For example, a “ManytoOne” relationship exists between Applications and Users, ensuring that each application is correctly mapped to its respective candidate.

3.3.3 Tables Created

User Table

- User ID
- Name
- Email
- Password
- Role

Job Table

- Job ID

- Job Title
- Description

Application Table

- Application ID
- Job ID
- Status

Resume Table

- Resume ID
- User ID
- Upload Date

Chapter 4

METHODOLOGIES

4.1 Project Architecture

The system uses a layered architecture where users access the application through a web browser connected to a React.js frontend. The frontend communicates with a Spring Boot backend via REST APIs, handling security, business logic, and data processing, while interacting with a MySQL database and integrating services like email notifications and chatbot.

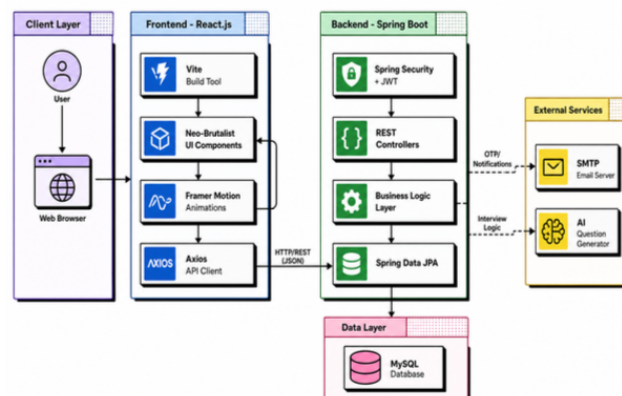


Figure 4.1: System Architecture Diagram

Figure 4.1 represents project architecture and how the system communicates.

4.2 DataFlow Diagram

The DataFlow Diagram represents the overall interaction between external entities and the Job Portal Management System. It shows three primary users: Student, Employer, and Admin, all interacting with a central system. Students provide credentials, profiles, and job applications, and receive job listings, application status, and notifications. Employers interact by posting jobs and managing candidates, while receiving applicant data and analytics. Admins manage system configuration and users, and receive reports and logs. This diagram highlights the high-level data flow and system boundaries without showing internal processing details.

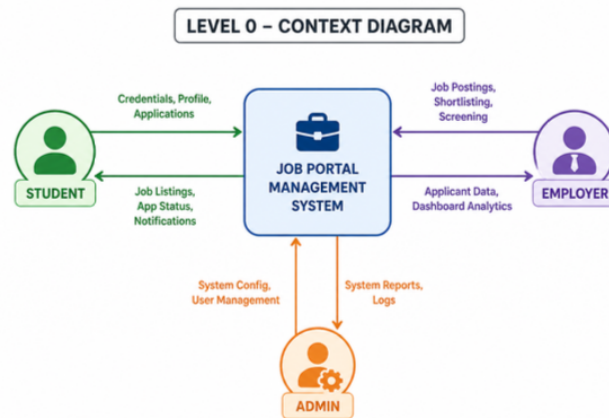


Figure 4.2: DataFlow Diagram

Figure 4.2 represents the dataflow diagram (level-0) DFD of our proposed job portal management system.

4.3 Module 1:User Management Module

User Management handles registration, login, and profile management with role-based access for students, employers, and admin.

4.4 Module 2:Job Management Module

Job Management includes job postings, applications, and candidate tracking between employers and students.

4.5 Module 3:Status Alert Module

Notification and Analytics provides email notifications, application updates, and basic system insights.

Advantages of this Project

- a) Neo-Brutalist UX:** Provides a high-contrast, modern interface that improves readability and user engagement.
- b) AI-Powered Efficiency:** Automates screening using AI-based question generation and chatbot support.
- c) Secure Architecture:** Ensures safe data handling through JWT-based authentication and role-based access.

Disadvantages

- a) Internet Dependency:** Requires a stable internet connection for full functionality.
- b) Server Load:** AI features increase backend computational and infrastructure requirements.

Chapter 5

RESULTS AND DISCUSSIONS

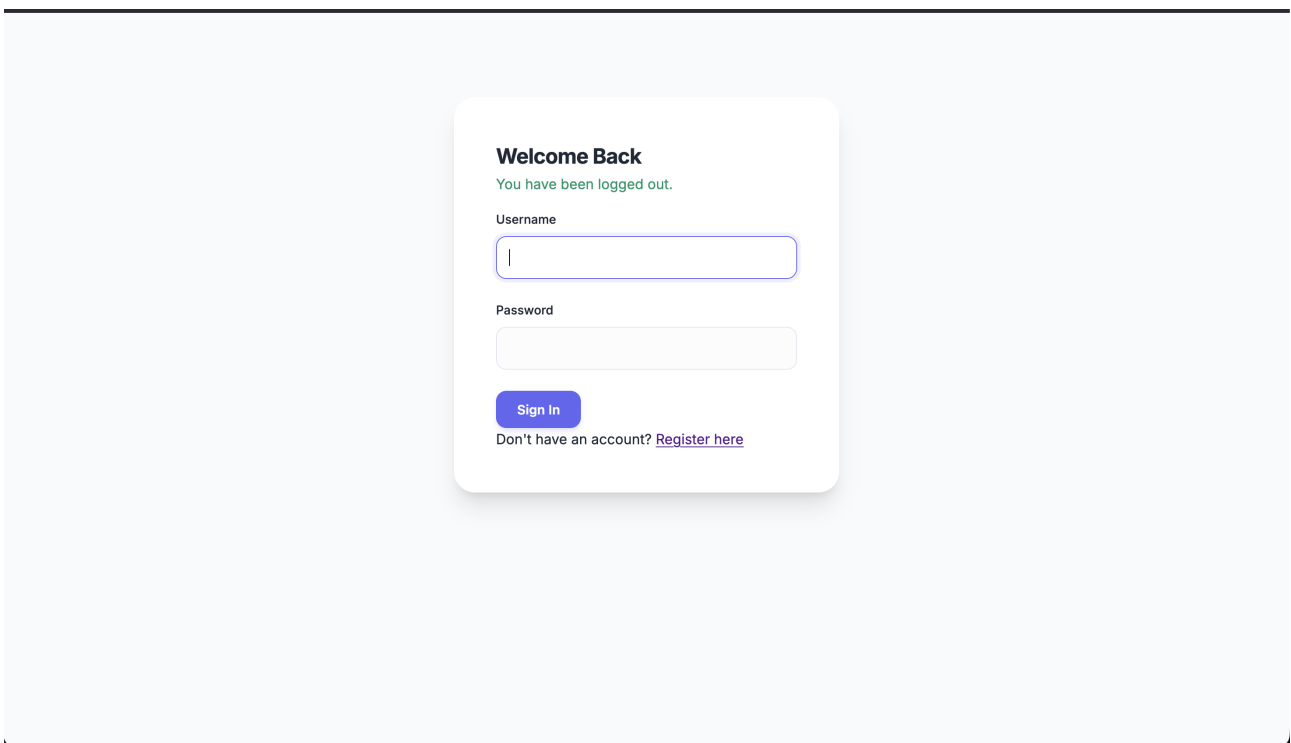


Figure 5.1: Login page for Employer and Student Authentication

The figure 5.1 represents and shows the login page where students (job applicants) and employers (job providers) can enter their credentials to access the system. Users can also log in via Google authentication (Firebase), and based on their roles, they are redirected to their respective dashboards.

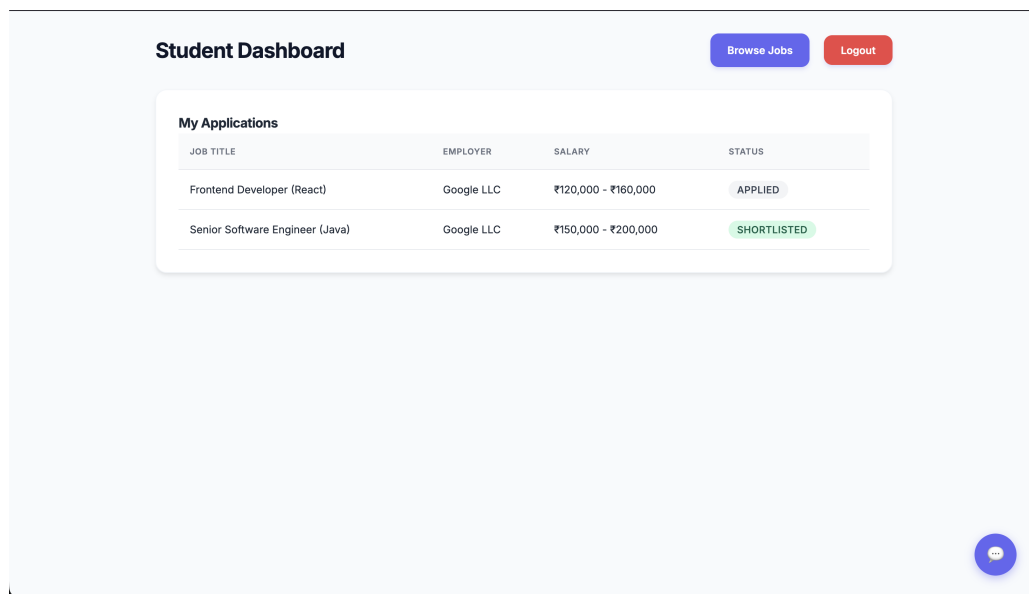


Figure 5.2: Student Dashboard

The figure 5.2 represents student dashboard (shown above) provides a centralized interface for managing all job-related activities efficiently. It enables users to track job applications in real time with clear status indicators such as applied, under review, shortlisted, and rejected, helping them monitor their progress easily. Students can upload and manage resumes, browse and apply for jobs based on their preferences, and update their profiles to improve visibility to employers. The dashboard also offers features like personalized job recommendations, notification alerts for application updates, and access to interview preparation tools. Additionally, an integrated chatbot assists users by providing instant guidance, answering queries, and offering suggestions to enhance their job search and application process. This interactive and user-friendly design ensures a smooth and productive experience for job seekers.

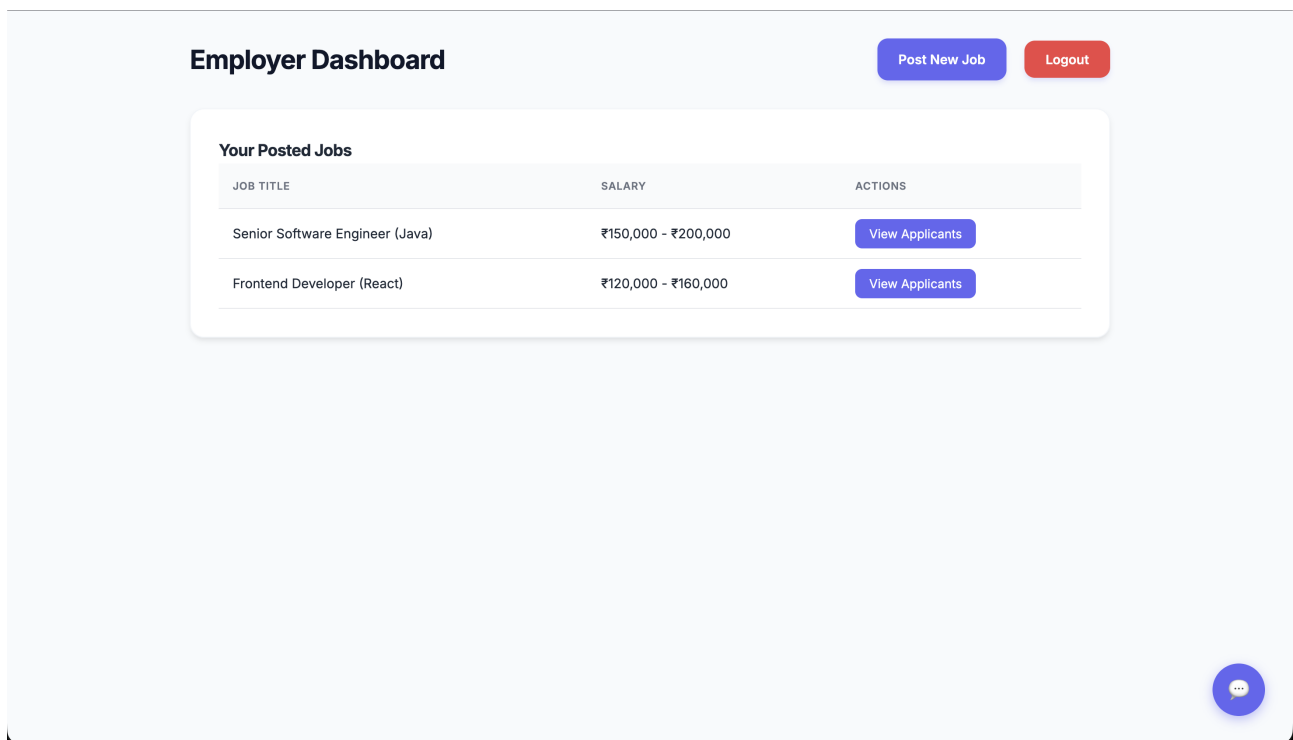


Figure 5.3: Admin Dashboard

The figure 5.3 represents the admin employer dashboard shown above provides a comprehensive overview of job postings and recruitment activities through an analytics-driven interface. It allows employers to create and manage job listings, monitor active and total job postings, and track the number of applications received. The dashboard presents key insights such as application trends over time, status distribution of candidates under review, shortlisted, rejected, and performance metrics like average applications per job, response rate, and shortlist rate. Employers can also communicate with applicants, manage company profiles, and post new job openings directly from the interface. This centralized system enables efficient hiring decisions by offering real-time data visualization, streamlined candidate management, and improved recruitment workflow.

Chapter 6

CONCLUSION AND FUTURE ENHANCEMENTS

6.1 Conclusion

The Job Portal Management System provides an efficient platform for connecting job seekers and employers through a modern, secure, and user-friendly interface. It simplifies the recruitment process with features like real-time tracking, role-based access, and AI support.

6.2 Future Enhancements

The system can be improved by adding features such as personalized job recommendations, mobile application support, advanced analytics, and video interview integration to enhance usability and scalability.

Chapter 7

ADDRESSING COMPLEX ENGINEERING PROBLEM AND SDG

7.1 Mapping of the Project to Complex Engineering Problem (CEP)

Level	Attribute	Characteristics	Wks	Justification
WP1	Depth of Knowledge	Requires in-depth engineering knowledge	WK3, WK4, WK5, WK6	Requires knowledge of Spring Boot, Spring MVC, Spring Security, JPA, and database design for building a multi-role job portal system
WP2	Conflicting Requirements	Technical and non-technical conflicts	WK4, WK5, WK6	Balances role-based access control with usability, ensuring employers and job seekers have separate secure and intuitive workflows
WP3	Depth of Analysis	Analytical and design thinking	WK3, WK4, WK5	It involves entity-relationship modeling, REST API design, and integration of file upload and email notification services
WP4	Familiarity	Partially new problems	WK4, WK8	Job-matching and application tracking introduce modern challenges in multi-role web systems beyond traditional CRUD applications
WP5	Applicable Codes	Limited standard solutions	WK6	Requires custom design for resume storage, application status tracking, and dynamic job filtering not fully covered by standard frameworks
WP6	Stakeholder Needs	Multiple stakeholders involved	WK5, WK6, WK7	Handles distinct requirements of job seekers, employers, and admins, ensuring secure and efficient role-specific interactions
WP7	Interdependence	System-level integration	WK3, WK5, WK6	Integrates frontend (Thymeleaf), backend (Spring Boot), database (JPA/MySQL), file storage, security, and email notification into one unified system

Table 7.1: Mapping of Project to Complex Engineering Problem (CEP)

7.2 Mapping of the Project to Complex Engineering Activities (CEA)

EA No.	Attribute	Characteristics	Justification based on the Project
EA1	Range of Resources	Use of diverse technologies	The project uses Spring Boot, Spring MVC, Spring Data JPA, MySQL/H2, Thymeleaf, Spring Security, and Java-MailSender for full-stack portal development
EA2	Level of Interactions	Complex interactions between modules	The system manages layered interactions between job listings, applications, user authentication, resume uploads, and email notifications
EA3	Innovation	Application of modern technologies	Integration of role-based dashboards, dynamic job filtering with JavaScript/ES6, file upload handling via Multi-partFile, and email status notifications demonstrates innovation
EA4	Consequences to Society and Environment	Impact on users and society	Improves employment accessibility, reduces manual hiring overhead, and provides a transparent and secure digital platform for job seekers and employers
EA5	Familiarity	Beyond standard practices	Combines multi-role authentication, resume management, application tracking, and real-time dashboard analytics into a unified portal beyond traditional job boards

Table 7.2: Mapping of Project to Complex Engineering Activities (CEA)

7.3 SDG Mapping (CEP Section)

SDG No.	SDG Title	Relevance to the Project
SDG 8	Decent Work and Economic Growth	Facilitates employment by connecting job seekers with employers, supporting fair recruitment processes and improving career opportunities
SDG 9	Industry, Innovation and Infrastructure	Promotes digital transformation in recruitment using Spring Boot REST APIs, dynamic filtering, and automated notification systems
SDG 10	Reduced Inequalities	Provides a standardised, accessible digital platform for all job seekers regardless of geography, reducing barriers to employment opportunities
SDG 16	Peace, Justice and Strong Institutions	Ensures data security and role-based access control using Spring Security and password hashing, maintaining integrity and fairness in the hiring process

Table 7.3: SDG Mapping for the Project

References

- [1] LinkedIn Corporation. (2024). LinkedIn Job Portal. [Online]. Available:
<https://www.linkedin.com/jobs/>
- [2] Indeed Inc. (2024). Indeed Job Search. [Online]. Available:
<https://www.indeed.com/>
- [3] Glassdoor Inc. (2024). Glassdoor. [Online]. Available:
<https://www.glassdoor.com/>
- [4] Info Edge (India) Ltd. (2024). Naukri.com. [Online]. Available:
<https://www.naukri.com/>
- [5] Pivotal Software. (2024). Spring Boot Reference Guide. [Online]. Available:
<https://docs.spring.io/spring-boot/docs/current/reference/html/>
- [6] Jakarta EE. (2024). Java Persistence API (JPA). [Online]. Available:
<https://jakarta.ee/specifications/persistence/>
- [7] United Nations. (2024). Sustainable Development Goals. [Online]. Available:
<https://sdgs.un.org/goals>